Assignment 1 — Two Clocks: Build a Bitemporal Database

Advanced Database Chapters and Cloud Databases - 2026

Summary

You will build a database that remembers two things about every fact: when it was true in the real world, and when the database believed it. Then you will prove it works by answering eight temporal questions, including one where the same question gets three different correct answers.

Item	Details
Goal	Design, populate and query a bitemporal database (valid time + transaction time) for a domain of your choice
RDBMS	Any: MySQL 8.4/9.x, MariaDB 11.x, PostgreSQL 17/18/19, Oracle 23ai Free, SQL Server 2022. State the engine and version in your README
Work mode	Individual
Deliverables	5 SQL scripts that run on an empty database, an ER diagram, a report of at most 5 pages
Grading	Core 8 points + Advanced track up to 2 points = 10
Deadline	Seminar 3, with a live run of your scripts during the seminar

The university course-enrollment example is off-limits; pick something else.

The story: one question, three correct answers

A hospital ward W1 is headed by Dr. Rusu. Dr. Pop takes over, but the paperwork reaches the database late, and the first version of it is wrong.

1. 2 Jan: the system goes live and records Rusu as head of W1.
2. 10 Mar: a clerk records that Pop took over on 1 Mar (late entry, 9 days after the fact).
3. 15 Mar: someone notices the handover was actually on 3 Mar and corrects it.

In an append-only bitemporal table these three events leave five rows. Nothing is overwritten; old beliefs are closed in transaction time, never deleted. The symbol ∞ means "still open".

Row	Head of W1	Valid time [from, to)	Transaction time [from, to)
1	Rusu	[2 Jan, ∞)	[2 Jan 09:00, 10 Mar 14:00)
2	Rusu	[2 Jan, 1 Mar)	[10 Mar 14:00, 15 Mar 11:00)
3	Pop	[1 Mar, ∞)	[10 Mar 14:00, 15 Mar 11:00)
4	Rusu	[2 Jan, 3 Mar)	[15 Mar 11:00, ∞)
5	Pop	[3 Mar, ∞)	[15 Mar 11:00, ∞)

Now ask: **who headed W1 on 2 March?**

- As the database knew it on 5 March: Rusu (row 1).
- As the database knew it on 12 March: Pop (row 3).
- As the database knows it today: Rusu (row 4).

All three answers are correct. A hospital auditor, a court, or a payroll department may each need a different one. Your database must be able to produce all of them, from a single query parameterised by *valid at* and *known at*.

Choose your domain

Pick one of the six domains below, or propose your own. Every domain has a **main population** (the many), a **central object** (the few), a **relationship** between them that starts, ends and restarts, and a **role holder** who changes over time.

#	Domain	Main population	Central object	Relationship (bitemporal)	Role that changes	Twists your data must contain
1	Football league	Players	Clubs	Contracts and loan spells	Head coach of a club	A transfer registered after the window closed; a loan cut short; a coach sacked and the date later disputed
2	Streaming platform	Subscribers	Plans and titles	Subscriptions; regional title licences	Plan price	A price change announced a month ahead; a cancellation backdated after a refund dispute
3	Car-sharing fleet	Members	Cars and stations	Rentals; car-to-station placement	Tariff per car class	A damage report entered days later; a car relocated and the move recorded to the wrong station
4	Apartment rentals	Tenants	Apartments and owners	Leases	Rent level; property manager	A renewal signed after the lease already restarted; a retroactive rent indexation
5	Airline crew	Crew members	Aircraft types and routes	Crew-to-flight assignments; type ratings	Chief pilot of a base	A rating that expired but was not recorded; a crew swap corrected after the flight
6	Esports league	Players	Teams and tournaments	Roster membership	Team captain; sponsor	A mid-season transfer; a roster change backdated after an appeal

Your own domain is accepted if it has at least five entities, at least three bitemporal tables, one relationship that can restart after a gap, one role that changes hands, and one numeric attribute that changes over time.

Modeling rules (apply to every bitemporal table)

These rules remove the guesswork. Follow them unless you justify a deviation in the report.

1. **Four period columns**, named consistently: `valid_from`, `valid_to`, `tx_from`, `tx_to`. Both periods are half-open, `[from, to)`, as in SQL:2011.
2. **Open end = sentinel, not NULL.** Use `9999-12-31` for valid time and `9999-12-31 23:59:59.999999` for transaction time. Sentinels keep predicates simple (`from <= x AND x < to`) and indexable. PostgreSQL users may use `daterange` / `tstzrange` with `infinity` instead.
3. **Types.** Valid time: `DATE`, unless your domain needs hours (then `DATETIME`/`TIMESTAMP`; say why). Transaction time: microsecond precision, stored in UTC. In MySQL use `DATETIME(6)`, not `TIMESTAMP`: `TIMESTAMP` stops at 2038-01-19 and cannot hold the sentinel.
4. **CHECK constraints** on every table: `valid_from < valid_to` and `tx_from < tx_to`.
5. **Anchor + versions.** Split each temporal entity into an anchor table (stable identity, e.g. `player(player_id, birth_date)`) and a version table (`player_version(version_id, player_id, …periods…)`). Ordinary foreign keys point at anchors.
6. **Transaction time is append-only.** Never `UPDATE` a business column, never `DELETE` a row. The only permitted update is setting `tx_to` on the current version. Every change is *close the old belief + insert the new belief*.
7. **One change, one timestamp.** All rows written by one logical change share the same `tx_from`. Keep a `tx_registry(tx_id, tx_time, author, reason)` table and store `tx_id` on every version row. This answers the lecture question of how to stamp a transaction that touches several tables.

8. **Two kinds of "delete".** *It stopped being true* (a player left the club) closes **valid** time: insert a new version whose `valid_to` is the end date. *It was never true* (a row entered by mistake) closes **transaction** time only, with no replacement.

Your data must also satisfy four invariants. You will check them with a view in Part C.

Id	Invariant	In plain words
I1	Temporal entity integrity	Among rows current in transaction time, the valid periods of one business key never overlap
I2	Temporal referential integrity	A child's valid period is covered by the union of its parent's valid periods (a contract cannot outlive the club)
I3	Single role holder	A role has at most one holder at any valid instant
I4	Immutable history	No row with `tx_to` in the past has ever been modified; `tx_from <= tx_to <= now` or `tx_to` = sentinel

Part A — Schema (1.5 points)

Deliver `01_schema.sql`: it creates everything from an empty database and can be re-run (`DROP … IF EXISTS` first).

- At least **5 entities**, at least **3 bitemporal version tables**: the relationship, the role assignment, and one entity with a changing attribute (price, salary, rating). Other tables may be non-temporal; say why each one is.
- Primary keys, foreign keys to anchors, `NOT NULL`, and the CHECK constraints from rule 4.
- The `tx_registry` table from rule 7.
- At least one **index designed for as-of queries** on your largest version table, for example `(business_key, tx_to, valid_from)`. In the report, show the `EXPLAIN` of Q6 with and without it.

A minimal version table looks like this (MySQL 8 syntax):

```
CREATE TABLE contract_version (
    version_id  BIGINT        NOT NULL AUTO_INCREMENT,
    player_id   INT           NOT NULL,
    club_id     INT           NOT NULL,
    salary_eur  DECIMAL(12,2) NOT NULL,
    valid_from  DATE          NOT NULL,
    valid_to    DATE          NOT NULL DEFAULT '9999-12-31',
    tx_from     DATETIME(6)   NOT NULL,
    tx_to       DATETIME(6)   NOT NULL DEFAULT '9999-12-31 23:59:59.999999',
    tx_id       BIGINT        NOT NULL,
    PRIMARY KEY (version_id),
    FOREIGN KEY (player_id) REFERENCES player (player_id),
    FOREIGN KEY (club_id)   REFERENCES club (club_id),
    FOREIGN KEY (tx_id)     REFERENCES tx_registry (tx_id),
    CHECK (valid_from < valid_to),
    CHECK (tx_from < tx_to),
    INDEX ix_contract_asof (player_id, tx_to, valid_from)
);
```

Part B — Data and the timeline (2 points)

Your data is not a static dump. It is a **replay of history**: the database goes live on day 1, then receives changes one transaction at a time, each with its own `tx_from`.

- `02_data.sql`: the initial load (event E1).
- `03_timeline.sql`: every later change, in transaction-time order, each block tagged with a comment such as `-- E3 late entry: Ionescu's contract, recorded 12 days late.`

Minimum volume, covering at least 12 months of valid time and two named periods (semesters, seasons, quarters):

Table role	Minimum rows
Main population (anchors)	50
Role holders	5
Central objects	6
Relationship facts (current, as known today)	60
Version rows across all bitemporal tables	120

The timeline must contain every event type below. Generated names and filler data are fine; the events must be designed by you.

Event	What happens	Signature in the data	Min. count
E1	Initial load	Many rows, one `tx_id`	1
E2	Proactive change, recorded before it takes effect	`valid_from > tx_from`	3
E3	Late entry, recorded at least 7 days after it happened	`tx_from - valid_from >= 7 days`	5
E4	Correction of a wrong value or wrong date	Old version closed in TT, replaced by a corrected one	3
E5	Real-world end (drop, cancel, leave)	New version with a finite `valid_to`	5
E6	Restart after a gap (re-join, re-enrol)	Two valid periods for the same pair with a gap between them	3
E7	Role holder changes, at least one mid-period and one later corrected	Role table on at least 2 central objects	3
E8	Erroneous insert retracted ("never true")	TT closed, no replacement	1
E9	One change touching two or more tables	Same `tx_id` in different tables	2

At least one E7 event must reproduce the pattern from *The story*: a question about a fixed valid date whose answer differs at three transaction dates.

Simulating the clock. You are loading history today, but `tx_from` must look like it was written in the past. Use a session-level simulated clock: `SET @sim_now = '2026-03-10 14:00:00';` in MySQL/MariaDB, or `SET app.sim_now = '…';` read with `current_setting()` in PostgreSQL. Every write takes its `tx_from` from that clock.

Finally, put a **story log** in your README: at least 10 of your events in plain language, each with its valid date and transaction date. They are used to check your query results.

Part C — Temporal questions (3 points)

Deliver `04_views.sql` with one view per question, and `05_tests.sql` that calls each view with concrete parameters and shows the output. Paste each output (or a screenshot) into the report.

Parameters. MySQL views cannot take arguments. Keep a one-row table `query_params(valid_at, known_at, period_start, period_end, subject_id)` that the views join to, and update it in `05_tests.sql`. In PostgreSQL or Oracle you may write SQL table functions instead.

Q	Question	Dimension	What makes it correct
Q0	`v_integrity_violations`: every row that breaks I1, I2 or I3	Both	Returns zero rows on your final data. Deliberately break one row in the test script to show it fires
Q1	Current facts for one member of the main population (e.g. a player's current contracts)	Both, now	Valid today **and** current in TT

Q	Question	Dimension	What makes it correct
Q2	Full history of a role holder for one central object, as known today, with the length of each tenure	VT	Adjacent periods with the same holder are **coalesced** into one row
Q3	Late entries, proactive entries and corrections, classified, with the delay in days and old vs new value side by side	TT vs VT	Every E2, E3 and E4 event from your story log appears, correctly labelled
Q4	Relationship facts valid during a named period, in two columns: *at some point* vs *throughout*	VT	Overlap for the first, containment for the second, both half-open
Q5	The whole relationship table as the database believed it at a transaction time ("What did the system know on 1 March?")	TT	Rows with `tx_from <= known_at < tx_to`, any valid time
Q6	Bitemporal point query: facts valid at `valid_at` as known at `known_at`	Both	Reproduces your three-answers case from Part B at three `known_at` values
Q7	Monthly count of active relationships per central object, as known today	VT	Uses a calendar (recursive CTE or table); months with zero still appear
Q8	Gaps between consecutive valid periods of the same pair (time between leaving and re-joining)	VT	Uses LAG/LEAD, as in the lecture's time-between-orders example; finds every E6 event

The core predicate you will reuse everywhere, for a table `t` and parameters `p`:

```
SELECT t.*
FROM   contract_version AS t
       CROSS JOIN query_params AS p
WHERE  t.valid_from <= p.valid_at
  AND  p.valid_at   <  t.valid_to
  AND  t.tx_from    <= p.known_at
  AND  p.known_at   <  t.tx_to;
```

Part D — ER diagram and report (1.5 points)

ER diagram (PDF or PNG, any tool: MySQL Workbench, dbdiagram.io, draw.io). Show anchors and version tables separately, mark each table as non-temporal, VT, TT or bitemporal, and use crow's-foot cardinalities.

Report, at most 5 pages, with these sections:

1. **Domain and why time matters** (half a page). Which real question in your domain needs both clocks?
2. **Modeling decisions.** Sentinel vs NULL, time granularity, anchor/version split, `tx_registry`, and any deviation from the modeling rules.
3. **Write protocol.** Show, as SQL, how you perform E3 (late entry), E4 (correction) and E8 (retraction). One short script each.
4. **Challenges and failed attempts.** Describe at least one approach you tried and abandoned (if any), and why it failed. Wrong turns documented honestly earn credit here.
5. **Insights.** Compare what you built by hand with what Oracle (Temporal Validity, Flashback), SQL Server (system-versioned tables) and PostgreSQL 18/19 (`WITHOUT OVERLAPS`, `FOR PORTION OF`) offer natively, based on the lectures. Which of your invariants would the engine have enforced for you?

Also include the `EXPLAIN` comparison from Part A and the query outputs from Part C.

Advanced track (up to 2 bonus points)

Each option is worth 1 point. Put the code in `06_advanced_a1.sql` and `06_advanced_a2.sql`.

A1 — The database manages transaction time itself (1 point)

No script may write `tx_from` or `tx_to` directly. Provide:

- **A write API**: stored procedures such as `sp_assert_fact`, `sp_end_fact`, `sp_correct_fact`, `sp_retract_fact`. Each opens one `tx_registry` entry, closes the old belief and inserts the new one, taking the time from the simulated clock when it is set and from `CURRENT_TIMESTAMP(6)` otherwise.
- **Guard triggers** that reject any direct `UPDATE` of a business column, any change to a closed row, and any physical `DELETE`.
- **A negative test script** that attempts each forbidden operation and shows the error.

MySQL hint: a trigger cannot insert into the table it is defined on (error 1442), so the close + insert logic belongs in the procedures, and the triggers only guard. PostgreSQL hint: an `INSTEAD OF` trigger on a view of current rows can do both.

A2 — Native temporal features (1 point)

Re-implement at least Q1, Q5 and Q6 using your engine's temporal syntax, and show they return the same rows as your core version. Suitable engines:

Engine	Valid time	Transaction time
PostgreSQL 18	Range columns, `PRIMARY KEY (…, valid WITHOUT OVERLAPS)`, `FOREIGN KEY (…, PERIOD valid)`	No native support: triggers or the `temporal_tables` extension
PostgreSQL 19 (release candidate due October 2026)	As PG 18, plus `UPDATE / DELETE … FOR PORTION OF`	As PG 18
MariaDB 11.x	`PERIOD FOR`, `WITHOUT OVERLAPS`, `FOR PORTION OF`	`WITH SYSTEM VERSIONING`, `FOR SYSTEM_TIME AS OF`
Oracle 23ai Free	Temporal Validity: `PERIOD FOR`, `AS OF PERIOD FOR`, `VERSIONS PERIOD FOR`	Flashback Data Archive: `AS OF TIMESTAMP`, `VERSIONS BETWEEN`
SQL Server 2022	Manual only	System-versioned tables, `FOR SYSTEM_TIME AS OF / FROM … TO / BETWEEN`

MySQL has no temporal syntax, so A2 requires one of these engines. Where the engine stamps transaction time itself and you cannot backdate it, explain in the report how you loaded your historical timeline and what you had to give up.

Grading

Core work is worth 8 points; the advanced track adds up to 2, for a maximum of 10.

Part	Criterion	Points
A	Schema follows the modeling rules; anchors, constraints, `tx_registry`, as-of index	1.5
B	All event types E1–E9 present, tagged and listed in the story log; minimum volumes met; simulated clock used	2.0
C	Q0–Q8 correct on your data (0.3 each, Q6 counts double)	3.0
D	ER diagram and report, including failed attempts and the `EXPLAIN` comparison	1.5
A1	Automatic transaction time with write API, guard triggers and negative tests	+1.0
A2	Native temporal features, results matching the core version	+1.0

Deductions:

- Scripts do not run in order on an empty database: fix it live in Seminar 3, or Part C is graded on what runs.
- Physical `UPDATE` or `DELETE` of history in the core track: −0.5 per affected table.
- An event type missing from the timeline: the matching question scores 0.

Live check in Seminar 3. You run scripts 01–05 from scratch, then answer one question chosen by the seminar lead, such as "Who held role X on date V, as the database knew it on date T?".

Submission

Load your solution in CABD Team folder named Assignment1 in a folder named <lastname_firstname> before Seminar 3:

```
lastname_firstname /
    README.md            engine + version, how to run, story log
    01_schema.sql
    02_data.sql
    03_timeline.sql
    04_views.sql
    05_tests.sql
    06_advanced_a1.sql   optional
    06_advanced_a2.sql   optional
    er_diagram.pdf
    report.pdf
```

Before you submit:

☐ Scripts 01–05 run in order on an empty database with no errors

☐ `v_integrity_violations` returns zero rows

☐ Every event type E1–E9 is tagged in `03_timeline.sql` and appears in the story log

☐ Q6 shows three different answers for one valid date at three transaction dates

☐ No `TIMESTAMP` column holds the sentinel (MySQL)

☐ The report names at least one failed attempt

Notes

If you use plain MySQL for the whole core track, you will build everything by hand, which is the point. For A2 you need another engine.

Don't use NULL for "still open" as NULL makes every predicate a special case and weakens index use. If you prefer NULL, justify it and handle it consistently.

You can generate the data: scripts, Faker or an AI assistant are fine for names and filler. The E1–E9 events and the story log must be your own design.

Use valid time in the future: E2 requires changes that are recorded before they take effect.

Valid time should have granularity of days, unless your domain truly needs hours (crew rosters, rentals). Transaction time is always microseconds.